Rethinking Source Control for Agentic Software Engineering

Architectural Transformation, Data Models, and Scalability Constraints in AI-Driven Development

Samir Baladi

Independent Interdisciplinary AI Researcher — Ronin Institute / Rite of Renaissance

Research Article — Systems Architecture / Software Engineering
Research Domain: Version Control Systems, Distributed Systems, Agentic Software Engineering
Status: Position Paper with Proposed Empirical Evaluation Protocol
DOI: https://doi.org/10.5281/zenodo.23934465
ORCID: 0009-0003-8903-0029 · gitdeeper11@gmail.com
November 2026

Abstract

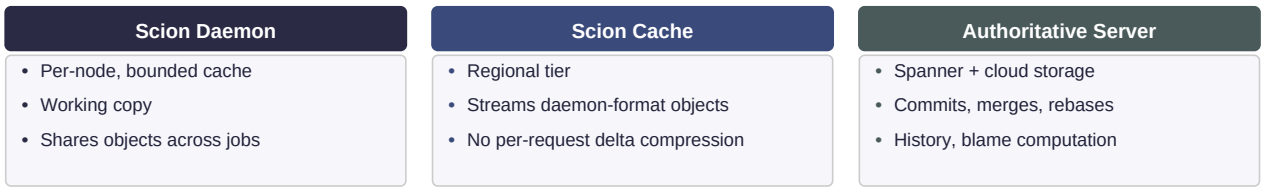
Source control systems were designed for a production model in which individual human engineers author, review, and merge changes at human pace. The emergence of autonomous coding agents capable of generating, testing, and submitting large volumes of parallel changes places a fundamentally different load on this model: higher change volume, higher parallelism, and a shift in the dominant cost center from network transfer to merge, review, and derived-data computation. In October 2026, Anthropic published a technical account of Scion, an internally developed source-control system intended to address these pressures, reporting operational figures including more than 30,000 merged changes per day, sub-second checkout operations, and a roughly 200-fold reduction in supporting infrastructure relative to a prior Git-mirroring architecture. This paper treats that disclosure not as a product announcement to be reproduced but as a case study motivating a broader architectural question: what structural transformations become necessary when code production shifts from a human-paced, sequential activity to a parallel activity driven by autonomous agents? Rather than asking whether Scion is superior to Git, the paper decomposes the problem into five independent axes — workload transformation, repository architecture, data architecture, execution infrastructure, and governance and security — and proposes a three-layer research design (architectural analysis, structural analysis, and empirical evaluation) for examining them. A detailed architectural analysis of Scion's changeset model, derived-data mechanism, and three-tier daemon/cache/server distribution is presented alongside two methodological corrections to a looser popular reading of the source material: Scion does not eliminate merge conflicts, and its documented absence of cross-server coordination applies specifically to the changeset-creation path rather than to the system as a whole. Open research questions are identified concerning the latency cost of on-demand derived-data computation and the scope of claimed functional decentralization. No independent benchmark results are reported in this manuscript; the operational figures discussed throughout are Anthropic's own published figures, and the empirical evaluation layer is presented as a proposed, not yet executed, research protocol.

Keywords: *Source Control; Version Control Systems; Agentic Software Engineering; Distributed Systems; Changesets; Derived Data; Repository Architecture; Scalability; Scion; Git.*

Evidentiary status: *Operational figures attributed to Scion (throughput, cache-hit rate, infrastructure footprint, checkout and CI improvements) are Anthropic's own published figures, not independently*

*reproduced results. · **Empirical status:** Layer C (reproducible empirical evaluation) is a proposed protocol; no experiments have been executed for this manuscript.*

Figure 1



Scion's Three-Tier Responsibility Distribution

Figure 1. The strict tier ordering — daemon, regional cache, authoritative server — is motivated by fault isolation as much as by performance: a regional cache failure does not flood the authoritative servers with direct traffic. Diagram constructed from the architectural description in Anthropic's October 2026 disclosure; not a reproduction of any original figure.

Figure 2

Five Independent Axes of Architectural Transformation

Figure 2. The research decomposes the general question into five axes that can be examined independently rather than collapsed into a single Git-versus-Scion comparison.

Table of Contents

1. Introduction	4
2. Related Work	6
3. Reframing the Research Question	8
4. Research Design: A Three-Layer Protocol	9
5. Architectural Analysis (Layer A)	10
6. Structural Analysis (Layer B)	12
7. Methodological Corrections to the Popular Reading	13
8. Open Research Questions	14
9. Proposed Empirical Evaluation (Layer C)	15
10. Discussion	16
11. Limitations	17
12. Conclusion	17
References	18
Research Status	19

1. Introduction

1.1 Background

Distributed version control systems such as Git were designed around an implicit production model: changes are authored by individual human engineers, committed locally, and merged at a pace bounded by human review and human typing speed. This model has proven durable for three decades of software engineering practice.

The emergence of autonomous coding agents — systems capable of generating, testing, and submitting code changes with limited human involvement in the loop — alters the assumptions underlying that model. Agents can generate and attempt to merge changes in parallel, at a volume and cadence that does not correspond to any historical human workload pattern.

In October 2026, Anthropic published a technical account of Scion, a source-control system developed internally to accommodate this shift, reporting substantial operational differences from a prior Git-based infrastructure, including higher daily merge throughput, smaller supporting infrastructure, and faster checkout operations (Anthropic, 2026).

1.2 The Problem With a Narrow Framing

A natural first reaction to this disclosure is to ask whether Scion is an improvement over Git, or to treat the published article as a specification to be reproduced or benchmarked directly. Both framings are limited.

A head-to-head comparison between two named systems risks collapsing a multidimensional architectural shift into a single verdict, and risks treating one vendor's internal engineering account as a controlled experiment rather than as a self-reported case study.

This paper adopts a different framing. The object of study is not Scion as a product, but the architectural transformation that systems serving agentic software engineering workloads appear to require — of which Scion is one documented instance, alongside Google's Piper and Meta's Sapling/Mononoke ecosystem.

1.3 Research Problem

The central problem addressed in this paper is:

> *What architectural changes become necessary when code production shifts from a predominantly human, sequential activity to a parallel activity driven substantially by autonomous agents?*

This framing deliberately does not presuppose that a centralized, client-server architecture is the only viable answer, and it does not restrict the inquiry to a single system.

1.4 Contribution and Scope

This paper makes four contributions:

1. A decomposition of the architectural transformation into five independent axes — workload, repository architecture, data architecture, execution infrastructure, and governance — that can be examined separately rather than conflated.
2. A detailed architectural analysis of Scion's changeset model, derived-data mechanism, and three-tier daemon/cache/server distribution, synthesized from the published source and clearly distinguished from independent verification.
3. Two explicit methodological corrections to claims that a looser reading of the source material might produce: that merge conflicts have been eliminated, and that the system is lock-free in an unqualified sense.

4 A proposed three-layer research protocol — architectural, structural, and empirical — for studying source-control architecture under agentic workloads, with the empirical layer presented as future work rather than completed research.

The paper does not claim to have independently reproduced any of Anthropic's reported performance figures. Where such figures are cited, they are explicitly attributed as self-reported.

DOI: https://doi.org/10.5281/zenodo.23934465

2. Related Work

2.1 Monorepo Infrastructure at Scale

Google's Piper system is the most extensively documented precedent for centralized, server-backed source control operating at very large scale. Potvin and Levenberg (2016) describe a single monolithic repository serving the majority of Google's engineering organization, backed by a custom distributed file system and supported by extensive tooling for code search, dependency analysis, and large-scale automated refactoring.

Piper predates the agentic-coding context examined in this paper, and its design responds primarily to organizational scale and cross-project dependency management rather than to agent-driven parallel change volume specifically. It nonetheless establishes that a centralized, non-Git architecture can serve engineering populations of a scale that distributed-clone models struggle to support.

2.2 Sapling and Mononoke

Meta's Sapling is a source-control client designed to present a Git-compatible user experience while interoperating with Mononoke, a server-side implementation optimized for very large repositories. Sapling emphasizes a fast, scalable commit graph and partial/virtualized checkouts, addressing performance problems that arise when Git's full-clone model is applied to monorepos of Meta's scale (Meta Engineering; Sapling SCM, n.d.).

Sapling and Mononoke are architecturally closer to Scion than Piper is, in that both pair a lightweight client with a more capable server-side system, and both adopt virtualized or sparse checkouts as a response to repository size. Neither system's public documentation centers agentic workloads as the primary design driver, though both are compatible with high change volume.

2.3 Distributed Database Infrastructure

Scion's authoritative server tier is reported to be backed by Spanner, Google's globally distributed database (Corbett et al., 2013). Spanner's externally consistent transaction model and global clock synchronization (TrueTime) are relevant background for understanding how a source-control system might achieve strong consistency for reference updates while distributing read-heavy traffic across a cache hierarchy.

This paper does not evaluate Spanner directly; it is cited because the architectural choice to back an authoritative source-control tier with a globally consistent database is itself a design decision with scalability and consistency implications distinct from Git's local, eventually-reconciled model.

2.4 Virtualized and Sparse Checkouts

Prior industrial work on making Git itself scale to very large repositories — including virtual file system approaches that materialize files on demand rather than requiring a full checkout — represents an alternative strategy to Scion's approach: extending Git's client-server boundary rather than replacing Git's data model outright. This line of work establishes that sparse or virtualized checkouts are not unique to non-Git systems, and that the checkout-latency problem Scion addresses has prior, Git-compatible partial solutions.

DOI: https://doi.org/10.5281/zenodo.23934465

2.5 Research Gap

The reviewed precedents establish that centralized, server-backed source control at scale is a well-studied engineering problem independent of autonomous agents. What is comparatively unexamined — and what Anthropic's 2026 disclosure newly foregrounds — is the specific load profile introduced by autonomous coding agents: high parallelism, high change frequency, and a shift in the dominant cost center from network transfer to merge, review, and derived-data computation.

This paper's contribution is not a claim that centralized source control is novel, but an attempt to isolate which architectural properties of systems such as Scion, Piper, and Sapling/Mononoke are specifically responsive to agentic workload characteristics, as distinct from properties that are simply responses to repository size or organizational scale.

3. Reframing the Research Question

3.1 Why Not "Is Scion Better Than Git?"

A direct comparison between Scion and Git invites several problems. First, the two systems are evaluated by their respective operators under different workloads, different organizational contexts, and different time horizons; a fair head-to-head comparison would require controlling for all three, which is not possible using only the published material. Second, a binary verdict obscures the fact that Scion's architecture responds to several at least partially independent pressures — raw change volume, checkout latency, history-query cost, and review governance — that a competing system might address differently without being simply "worse."

Third, and most importantly for this paper's purpose, the question of which system is better is not the question with the greatest research value. The more generative question is what the shift from human-paced to agent-paced code production implies for source-control architecture in general, independent of any single vendor's implementation.

3.2 Proposed Research Question

What architectural, data-model, and infrastructure changes become necessary — and which remain optional design choices — when source control must support a substantial fraction of changes originating from parallel, autonomous agents rather than sequential human authorship?

3.3 Five Independent Axes

The research question is decomposed into five axes, each of which can in principle be studied, and in some cases addressed, independently of the others.

Axis	Focus
01 — Workload Transformation	Change volume, generation rate, parallelism, and merge frequency under agentic authorship
02 — Repository Architecture	Shift from a fully distributed repository model toward horizontally scaled, server-backed services
03 — Data Architecture	Changesets, write-ahead logs, content addressing, and derived data
04 — Execution Infrastructure	Sparse and virtualized checkouts, tiered caching, server-side APIs
05 — Governance and Security	Intent-based review, directory-level IAM, post-merge testing, and concurrent-conflict handling

Treating these as separate axes allows a later finding — for example, that sparse checkouts are necessary while a fully centralized data model is not — to be stated precisely, rather than folded into an undifferentiated judgment about "the new architecture" as a single package.

3.4 Proposed Title and Framing

Consistent with this reframing, the paper is titled to avoid presupposing the research outcome: *Rethinking Source Control for Agentic Software Engineering* neither assumes that a centralized architecture is the unique solution nor restricts the inquiry to Scion specifically.

DOI: https://doi.org/10.5281/zenodo.23934465

4. Research Design: A Three-Layer Protocol

The study is structured as three layers, intended to be pursued in sequence, with later layers depending on findings from earlier ones.

Layer	Focus	Status in this manuscript
A — Architectural Analysis	Detailed architectural analysis of Git, Scion, Piper, and Sapling (Section 5)	Presented
B — Structural Analysis	Data, storage, concurrency, consistency, and scalability properties	Presented (Section 6)
C — Empirical Evaluation	Reproducible, controlled tests measuring the effect of agent-native workload patterns (Section 9); system not executed	Proposed protocol only

The separation between Layers A/B and Layer C is deliberate and load-bearing for the epistemic status of this manuscript. Sections 5 and 6 synthesize and critically examine published architectural descriptions; they do not report new measurements. Section 9 specifies an evaluation protocol capable of producing new measurements, but no such measurements have yet been collected.

This separation also governs how Anthropic's published operational figures are treated throughout the paper: as Layer A material — architectural self-description together with self-reported operational outcomes — not as Layer C findings.

5. Architectural Analysis (Layer A)

5.1 Git and Scion: A Structural Comparison

Anthropic's published account frames Scion's design around seven structural departures from Git's model, summarized in Table 2.

Property	Traditional Git	Scion
Topology	Distributed	Client–server
Connectivity assumption	Offline-first	Always-online
Checkout model	Full repository clones	Sparse / virtualized checkouts
Object storage	Packfiles	Content-addressed objects
Commit locus	Client-side commits	Server-side changesets
Reference model	Repository refs	Reduced refs
Operational model	Repository maintenance (gc, repack)	Horizontally scalable services

The significance of this comparison, for the purposes of the present study, is not that each row represents an isolated optimization, but that the seven changes co-occur: Scion redistributes responsibility between client and server, caching layers, and derived-data computation as a coordinated architectural package rather than as independent patches to Git's model.

5.2 The Changeset as the Redefined Unit of Work

In Scion, a changeset is not simply a differently named commit; it redefines the unit of work itself.

- **Definition:** a changeset is a record in a write-ahead log (WAL) describing the files changed relative to the first parent only, together with content-addressed pointers into blob storage for the compressed file contents.
- **The critical architectural separation:** changeset creation is fully decoupled from reference updates. This separation is what allows changes to be accepted without requiring full cross-server coordination at creation time.
- **Absence of pre-built tree structures:** unlike Git, a changeset does not store a complete tree structure; the server derives tree structures later, on demand or eagerly, rather than requiring the client to construct and transmit them.
- **Discouragement of long-lived branches:** because the global state that all servers must agree on becomes a scaling bottleneck as the number of concurrent branches grows, Scion's design discourages branch proliferation in favor of narrower, shorter-lived units of change.

5.3 Derived Data as an Architectural Primitive

A central claim examined in this paper is that Scion's handling of derived data is not merely a storage or caching optimization layered on top of a conventional data model, but a structural decision that separates the canonical change log from the query-serving structures built from it.

- **Mechanism:** for operations such as blame, path history, and commit-graph traversal, Scion builds additional derived structures, computed recursively from parent changesets, either eagerly or on demand.

DOI: https://doi.org/10.5281/zenodo.23934465

- **Architectural payoff:** because this derivation happens server-side, new categories of derived data can be introduced or optimized without requiring any client-side change. This fully decouples the base data model from the system's query capabilities.

This property is architecturally significant because it inverts a common assumption in version-control design — that the representation used for storage and the representation used for history queries must be the same structure, or must evolve in lockstep on the client. Scion's design instead treats the write path (the changeset log) and the read path (derived structures) as independently evolvable.

5.4 Three-Tier Responsibility Distribution

Scion's runtime is organized into three tiers, each with a distinct role and a distinct failure-isolation purpose.

Scion Daemon (per node)

Maintains a bounded local cache and the working copy. Its principal function is to share cached objects across multiple jobs running on the same node, avoiding redundant fetches for closely timed, co-located work.

Scion Cache (regional)

Streams objects directly in the daemon's native storage format, eliminating the need for per-request delta compression and reducing network-layer processing cost.

Authoritative Server

Backed by Spanner and cloud object storage, this tier performs the computationally heavy operations: commits, merges, rebases, history computation, and blame.

The strict ordering of these tiers — daemon, then regional cache, then authoritative server — is motivated by failure isolation as much as by raw performance: a regional cache outage does not cause client traffic to flood the authoritative servers directly, because the daemon and cache layers absorb and shape the request pattern before it reaches the server tier.

6. Structural Analysis (Layer B)

6.1 Data Model Properties

Four structural properties distinguish Scion's data model from Git's object model in ways relevant to agentic workloads:

1. **Write-ahead, parent-relative diffs** rather than full-tree snapshots per commit, reducing the marginal cost of each additional changeset.
2. **Content addressing** for blob storage, enabling deduplication across changesets and clients without requiring a shared working history.
3. **Separation of write path and read path**, as discussed in Section 5.3, which allows query-serving structures to scale and evolve independently of the append-only log.
4. **Reduced reference surface**, limiting the set of mutable pointers that must be kept globally consistent, which directly addresses the branch-proliferation scaling problem noted in Section 5.2.

6.2 Consistency and Coordination

Not every operation in Scion has the same coordination requirements. Changeset creation, as described in Section 5.2, is designed to avoid full cross-server coordination. Reference updates are a different matter: they are reported to rely on compare-and-swap (CAS) semantics, which is a form of optimistic concurrency control requiring the updating party to detect and retry on conflicting concurrent updates.

This distinction is structurally important: a system can be decoupled from global coordination on one operation path while retaining coordinated, serializable semantics on another. Treating the system as uniformly "coordination-free" would overstate what the architecture actually guarantees.

6.3 Scalability Properties

The reported operational figures — in excess of 30,000 merged changes per day as of September 2026, a greater than 99% cache hit rate at every layer, and an approximately 200-fold reduction in supporting infrastructure relative to a prior Git-mirroring approach — are presented by Anthropic as outcomes of the architecture described above (Anthropic, 2026).

From a structural-analysis standpoint, three properties plausibly contribute to this scalability profile: (i) the decoupling of changeset creation from reference coordination, which removes a global synchronization point from the highest-frequency operation; (ii) the daemon/cache/server tiering, which absorbs read traffic before it reaches the authoritative tier; and (iii) the content-addressed, derived-data model, which avoids redundant recomputation of history-dependent structures across clients.

This structural account does not constitute independent verification of the reported figures. It is an analysis of which architectural properties would be expected, on general distributed-systems grounds, to produce the kind of scaling behavior described, and is offered as a basis for the testable predictions proposed in Section 9.

DOI: https://doi.org/10.5281/zenodo.23934465

7. Methodological Corrections to the Popular Reading

Three points require explicit correction relative to a looser, more enthusiastic reading of Scion's published description. These corrections were identified during the development of this manuscript and are treated as binding constraints on all other claims made in this paper.

7.1 Scion Does Not Eliminate Merge Conflicts

The published account does not describe Scion as having removed the possibility of conflicting changes. Rather, Anthropic reports that it was not able to retain a traditional global merge-queue model at the observed change volume, and instead relies more heavily on post-merge and pre-release testing, accepting that conflicts can occur and be identified and corrected after the fact rather than exclusively gated before merge.

This is a different claim from "conflicts no longer occur," and the distinction matters for any evaluation of correctness guarantees: a system that defers conflict detection to post-merge testing trades earlier gating for higher merge throughput, which is a tradeoff with real correctness and debugging-cost implications, not a strictly dominant improvement.

7.2 Scion Is Not Unconditionally "Lock-Free"

The published material describes changes and merges as scaling without locks or full cross-server coordination specifically along the changeset-creation path. It does not claim the absence of coordination mechanisms throughout the system. As noted in Section 6.2, reference updates are reported to use compare-and-swap, an optimistic concurrency-control mechanism that is itself a form of coordination, even though it does not require a global lock.

Describing the system as lock-free without this qualification would misrepresent the architecture and would obscure a genuinely interesting structural finding: that different operation types within the same system can have different coordination requirements, and that this differentiation — rather than the blanket absence of coordination — may be the more transferable architectural lesson.

7.3 Derived Data Is an Architectural Decision, Not a Storage Optimization

As discussed in Section 5.3, treating derived data as "just caching" understates its role. The separation between the canonical changeset log and the structures derived from it for history, blame, and path-history retrieval is a data-architecture decision with consequences for client-server coupling, independent evolvability of query capabilities, and — as examined in Section 8.1 — potential latency tradeoffs under on-demand computation.

DOI: https://doi.org/10.5281/zenodo.23934465

8. Open Research Questions

Beyond the architectural description in Sections 5 and 6, two questions are identified as open and not resolved by the published source material.

8.1 The Hidden Cost of Separating Data From Query

If derived data such as blame and path history is computed on demand rather than eagerly, what is the effect on response latency for complex historical queries under high concurrent load? Is there an implicit tradeoff between the simplicity of the base data model (a thin, append-only changeset log) and the latency of history-dependent queries, and if so, how is that tradeoff managed as repository history grows?

This question cannot be answered from the published architectural description alone. It is proposed as a candidate hypothesis for the empirical evaluation protocol in Section 9.

8.2 The Limits of Functional Decentralization

Scion depends on centralized authoritative servers for its heaviest computational operations. This implies that system availability and fault tolerance depend substantially on the design and redundancy of those servers, in a way that a fully distributed model such as Git's does not.

A related question is one of scope: does the claim of operating "without full cross-server coordination" apply only to the changeset-creation path, as argued in Sections 5.2 and 7.2, while other operations — reference updates via compare-and-swap, merges, and history computation — continue to depend on coordinated, server-side processing? If so, "decentralization" in Scion's design is better understood as a property of a specific write path rather than as a property of the system as a whole, and claims of decentralization should be scoped accordingly in any future comparative work.

DOI: https://doi.org/10.5281/zenodo.23934465

9. Proposed Empirical Evaluation (Layer C)

> ***Status:*** *Proposed protocol only. No experiments have been executed for this manuscript.*

Layer C is intended to translate the open questions of Section 8 and the structural claims of Section 6 into reproducible, controlled measurements. It is specified here at the level of design, not results.

9.1 Candidate Measurements

- Checkout latency as a function of repository size, under sparse/virtualized versus full-clone configurations.
- Merge throughput and conflict-detection timing under synthetically generated agentic workload patterns (high parallelism, short-lived branches, high change frequency).
- History-query latency (blame, path history) under on-demand versus eager derived-data computation, at varying history depth and concurrent load — directly targeting the question raised in Section 8.1.
- Cache-hit rate at each tier (daemon, regional cache, authoritative server) under controlled request-mix scenarios.
- Failure-isolation behavior: the effect of a simulated regional-cache outage on authoritative-server load, relative to the architectural claim in Section 5.4.

9.2 Candidate Systems Under Test

A controlled comparison would require either access to Scion itself, which is not publicly available at the time of writing, or a comparison restricted to publicly accessible systems exhibiting structurally similar properties, such as Git with virtualized-checkout extensions, Sapling/Mononoke, and a conventional Git-mirroring baseline.

Where Scion itself cannot be benchmarked directly, this limitation must be stated explicitly in any resulting empirical report, and claims about Scion's actual performance must continue to be attributed to Anthropic's own disclosure rather than to independently collected data.

9.3 Workload Generation

A synthetic agentic-workload generator is proposed, parameterized by change rate, parallelism (number of concurrent in-flight changesets), change size distribution, and merge-conflict injection rate, to allow controlled, repeatable comparison across systems rather than reliance on organic production traffic that cannot be shared or replayed.

9.4 Preregistration and Reporting Discipline

Consistent with the evidentiary discipline established in Section 7, any future execution of this protocol should distinguish pre-specified (confirmatory) measurements from exploratory ones, should report null or negative results with the same prominence as positive ones, and should not present any finding as a reproduction of Anthropic's reported figures unless the experimental conditions are shown to be comparable.

DOI: https://doi.org/10.5281/zenodo.23934465

10. Discussion

10.1 What Appears Load-Bearing Versus What Appears Optional

Synthesizing Sections 5 through 8, a tentative distinction can be drawn between architectural properties that appear necessary for supporting high-volume agentic workloads and properties that appear to be design choices made within that constraint rather than consequences of it.

The decoupling of changeset creation from reference-update coordination (Section 5.2), and the separation of the write path from derived query structures (Section 5.3), both plausibly address a load profile characterized by high parallelism and high change frequency, independent of which specific system implements them. By contrast, the choice to centralize the authoritative tier behind a single globally consistent database (Section 2.3) is one way to provide strong consistency for reference updates, but the architectural analysis in this paper does not establish that it is the only viable way; a more federated consistency model remains an open design alternative not ruled out by the evidence reviewed here.

10.2 Governance Implications

The shift toward post-merge testing rather than exclusively pre-merge gating (Section 7.1) has governance implications beyond raw throughput. Intent-based review, directory-level identity and access management, and automated post-merge validation become more load-bearing for correctness when pre-merge conflict gating is relaxed. This suggests that governance and security (Axis 05 in Section 3.3) cannot be treated as a peripheral concern layered on top of a purely technical architecture; it is a necessary counterpart to the throughput gains described in Sections 5 and 6.

10.3 Relation to Prior Monorepo Infrastructure

Section 2 situates Scion relative to Piper and Sapling/Mononoke. The present analysis suggests that what is specifically new in the agentic context is not the centralized, server-backed architecture itself — which Piper demonstrates can scale to very large human engineering organizations — but the particular combination of extremely short-lived units of change, very high parallelism, and reduced reliance on pre-merge human review, which together place different pressure on the coordination and derived-data layers than human-paced monorepo usage does.

DOI: https://doi.org/10.5281/zenodo.23934465

11. Limitations

Several limitations apply to this manuscript.

1. The architectural analysis in Sections 5 and 6 is synthesized from a single public disclosure (Anthropic, 2026) and from publicly available documentation of comparator systems; it has not been verified against Scion's source code or internal design documents, which are not publicly available.
2. All operational figures attributed to Scion throughout this paper are Anthropic's own published figures. No independent benchmarking has been performed, and the magnitude, measurement methodology, and workload conditions behind those figures are not independently verifiable from the source material.
3. The proposed empirical evaluation protocol (Section 9) has not been executed. No claims in this paper should be read as reporting results from that protocol.
4. The comparison with Piper and Sapling/Mononoke relies on each system's own public documentation, which may differ in depth, currency, and candor from the others, limiting the precision of cross-system comparison.
5. The paper's five-axis decomposition (Section 3.3) is a proposed analytical framework, not a validated taxonomy; its usefulness for organizing future research remains to be assessed against alternative decompositions.

12. Conclusion

Anthropic's 2026 disclosure of Scion documents a substantial architectural departure from Git, motivated by a production context in which autonomous coding agents generate a large share of code changes in parallel. This paper has argued that the most productive research framing of this development is not a binary comparison between Scion and Git, but an inquiry into which architectural properties — changeset/reference decoupling, separation of write and read paths through derived data, tiered caching with failure isolation, and governance mechanisms adapted to deferred conflict resolution — are structurally responsive to agentic workload characteristics as such, independent of any single vendor's implementation.

Two corrections to a looser popular reading of the source material have been made explicit: Scion does not eliminate merge conflicts, and its documented freedom from cross-server coordination applies to the changeset-creation path specifically rather than to the system as a whole. Two open research questions have been identified — the latency cost of on-demand derived-data computation, and the precise scope of claimed functional decentralization — that are not resolved by the published material and are proposed as targets for the empirical evaluation protocol outlined in Section 9.

This paper has not independently verified Anthropic's reported operational figures, and the proposed empirical layer remains unexecuted. The contribution offered here is architectural and conceptual: a reframed research question, a five-axis decomposition, a structural analysis of a publicly documented system, and a reproducible protocol for testing the claims that analysis generates. Whether the architectural properties identified here generalize beyond Scion to other systems serving agentic software engineering workloads is a question for the empirical research this paper proposes but does not itself conduct.

DOI: https://doi.org/10.5281/zenodo.23934465

References

1. Anthropic. (2026). **Redesigning Source Control at Anthropic.** Anthropic Research, published October 13, 2026 (updated October 15, 2026). https://www.anthropic.com/research/redesigning-source-control

2. Potvin, R., & Levenberg, J. (2016). **Why Google Stores Billions of Lines of Code in a Single Repository.** Communications of the ACM, 59(7), 78–87. https://doi.org/10.1145/2854146

3. Corbett, J. C., Dean, J., Epstein, M., et al. (2013). **Spanner: Google's Globally-Distributed Database.** ACM Transactions on Computer Systems, 31(3), Article 8. https://doi.org/10.1145/2491245

4. Sapling SCM. (n.d.). **Sapling: A Scalable, User-Friendly Source Control System.** Meta Platforms. https://sapling-scm.com/

5. Meta Platforms. (n.d.). **Sapling / Mononoke Source Repository.** GitHub. https://github.com/facebook/sapling

6. Chacon, S., & Straub, B. (2014). **Pro Git** (2nd ed.). Apress. https://git-scm.com/book

References 2, 3, and 6 describe prior, independently published systems (Piper, Spanner, and Git itself) used here for architectural comparison; they are not descriptions of Scion. Reference 1 is the sole source for all Scion-specific architectural and operational claims in this paper.

Research Status

Component	Current status
Research question	Reframed and specified (Section 3)
Five-axis decomposition	Proposed analytical framework
Layer A — Architectural analysis	Presented (Section 5)
Layer B — Structural analysis	Presented (Section 6)
Methodological corrections	Applied throughout (Section 7)
Open research questions	Identified, not resolved (Section 8)
Layer C — Empirical evaluation	Protocol proposed; not executed
Independent benchmarking of Scion	Not performed; not currently possible (system not public)
Operational figures cited	Anthropic's own published figures only

Record ID: RSC-AGENTIC-01 · Version: 1.0 · DOI: https://doi.org/10.5281/zenodo.23934465
Scope: Architectural position paper with proposed empirical evaluation protocol · Empirical baseline: Not yet executed. Primary source: Anthropic (2026), Redesigning Source Control at Anthropic, https://www.anthropic.com/research/redesigning-source-control
Next Research Stage: Execution of the Layer C evaluation protocol against publicly accessible comparator systems, and preregistration of confirmatory measurements prior to data collection.

Samir Baladi · Independent Interdisciplinary AI Researcher — Ronin Institute / Rite of Renaissance · ORCID: 0009-0003-8903-0029 · gitdeeper11@gmail.com